

INFORME DE LABORATORIO

Transforma texto en embeddings

STIVEN DAVID ÁLVAREZ OLMOS

608144/PROCESAMIENTO DE LENGUAJE
NATURAL/FU/CAD2202023206/EIAIIPA2026_2

septiembre de 2026

1. Introducción

Las computadoras no entienden las palabras tal como las leemos nosotros; para poder procesar texto, necesitan traducir todo a números. De eso se encarga el Procesamiento del Lenguaje Natural (NLP).

En este laboratorio trabajé en la conversión de texto a números: primero separando las frases en *tokens* y luego convirtiendo esos tokens en vectores mediante una capa de *embeddings*. Para poner esto a prueba, armé un clasificador de sentimientos sobre reseñas de películas usando TensorFlow y Keras. La idea era ver cómo funciona todo este flujo técnico y comprobar qué tan bien responde la red neuronal cuando se le presentan datos nuevos.

2. Objetivos

Objetivo general: Construir un modelo en TensorFlow que convierta texto en vectores mediante embeddings para clasificar el sentido de distintas opiniones.

Objetivos específicos:

- Entender cómo funciona la tokenización y la conversión de palabras a números.
- Implementar y configurar una capa de Embedding en TensorFlow.
- Entrenar una red neuronal para hacer clasificación binaria (positivo / negativo).
- Evaluar el rendimiento fijándome en la precisión y en la función de pérdida.
- Analizar e interpretar los resultados y las gráficas del entrenamiento.
- Dejar el código documentado y subido en un repositorio de GitHub.

3. Fundamento teórico

Un *embedding* asigna a cada palabra un vector de números reales, permitiendo que la red aprenda relaciones entre ellas.

Texto → Tokens → Índices → Embeddings (32D) → Red Neuronal → Predicción

4. Desarrollo del laboratorio

Usé el dataset IMDB de TensorFlow (10,000 palabras de vocabulario, máximo 200 tokens por reseña).

Arquitectura:

1. Embedding (32 dimensiones).
2. GlobalAveragePooling1D (para resumir la secuencia).
3. Capa Densa (32 neuronas, ReLU).
4. Dropout (evita *overfitting*).
5. Salida (Sigmoide para resultado binario).

Entrené el modelo durante 5 épocas y lo evalué con el conjunto de prueba.

.

5. Resultados obtenidos

Al ejecutar el notebook y evaluar el modelo con los datos de prueba, obtuve los siguientes números:

Métrica	Valor
Pérdida en prueba (Loss)	0.3306
Precisión en prueba (Accuracy)	0.8609
Precisión en porcentaje	86.09%

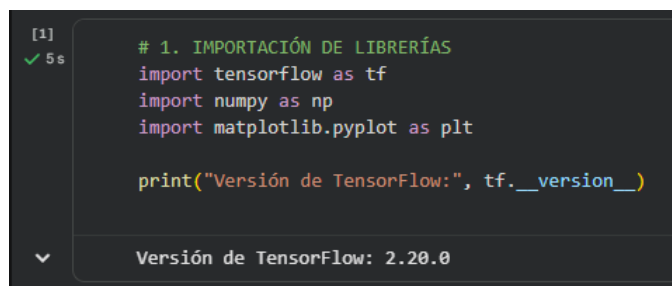
El modelo le acertó al 86.09% de las reseñas que no había visto antes. El valor de pérdida de 0.3306 refleja qué tan alejadas estuvieron sus probabilidades numéricas respecto a las etiquetas reales.

6. Análisis de resultados

Un 86% de acierto confirma que los *embeddings* capturaron bien la intención de los textos. La pérdida de 0.3306 indica predicciones con buena certeza. Aún así, la red no "entiende" el lenguaje como un humano y falla ante el sarcasmo o reseñas con opiniones mixtas. El resultado responde directamente a la configuración usada (vocabulario, longitud de texto y épocas).

7. Evidencias del Notebook (*Capturas de Google Colab*)

- **Figura 1.** Carga de librerías y preprocesamiento de las reseñas de IMDB.



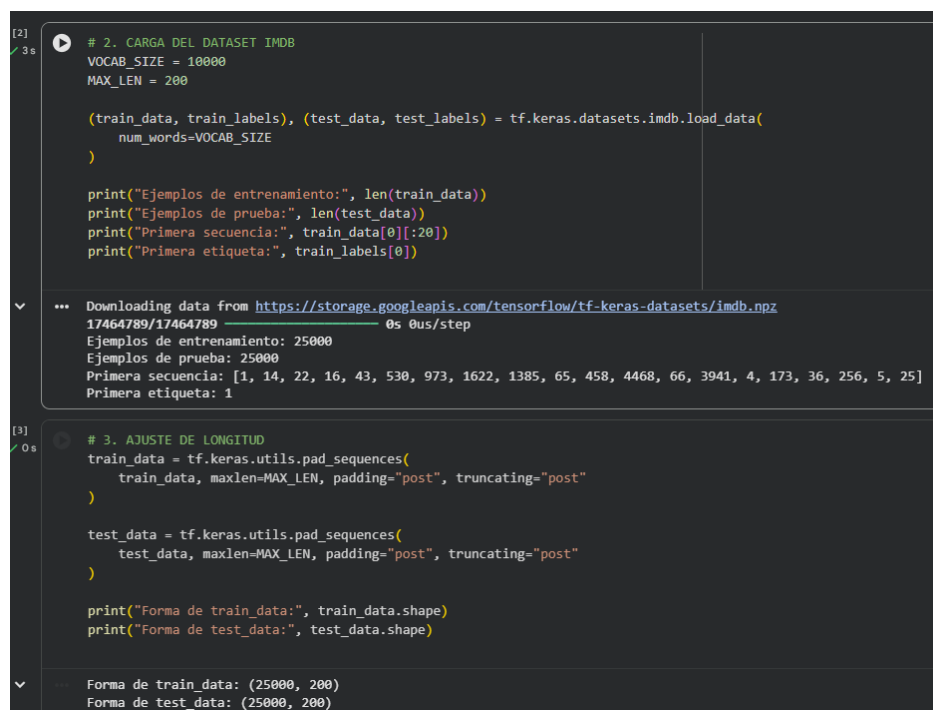
```
[1]
✓ 5 s

# 1. IMPORTACIÓN DE LIBRERÍAS
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("Versión de TensorFlow:", tf.__version__)

Versión de TensorFlow: 2.20.0
```

- **Figura 2.**



```
[2]
✓ 3 s

# 2. CARGA DEL DATASET IMDB
VOCAB_SIZE = 10000
MAX_LEN = 200

(train_data, train_labels), (test_data, test_labels) = tf.keras.datasets.imdb.load_data(
    num_words=VOCAB_SIZE
)

print("Ejemplos de entrenamiento:", len(train_data))
print("Ejemplos de prueba:", len(test_data))
print("Primera secuencia:", train_data[0][:20])
print("Primera etiqueta:", train_labels[0])

... Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 0s 0us/step
Ejemplos de entrenamiento: 25000
Ejemplos de prueba: 25000
Primera secuencia: [1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468, 66, 3941, 4, 173, 36, 256, 5, 25]
Primera etiqueta: 1

[3]
✓ 0 s

# 3. AJUSTE DE LONGITUD
train_data = tf.keras.utils.pad_sequences(
    train_data, maxlen=MAX_LEN, padding="post", truncating="post"
)

test_data = tf.keras.utils.pad_sequences(
    test_data, maxlen=MAX_LEN, padding="post", truncating="post"
)

print("Forma de train_data:", train_data.shape)
print("Forma de test_data:", test_data.shape)

Forma de train_data: (25000, 200)
Forma de test_data: (25000, 200)
```

Se utiliza el conjunto de datos **IMDB**, incluido en `tf.keras.datasets`. Contiene reseñas de películas etiquetadas como:

- 0: sentimiento negativo
- 1: sentimiento positivo

Para facilitar el experimento, se limita el vocabulario a 10.000 palabras y cada reseña se lleva a una longitud máxima de 200 tokens.

- **Figura 3.** Resumen del entrenamiento

Se divide una parte de los datos de entrenamiento para validación. La métrica principal será accuracy, mientras que loss permite observar el error del modelo.

```
[6]
✓ 11s # 6. ENTRENAMIENTO
      history = model.fit(
          train_data,
          train_labels,
          validation_split=0.2,
          epochs=5,
          batch_size=128,
          verbose=1
      )

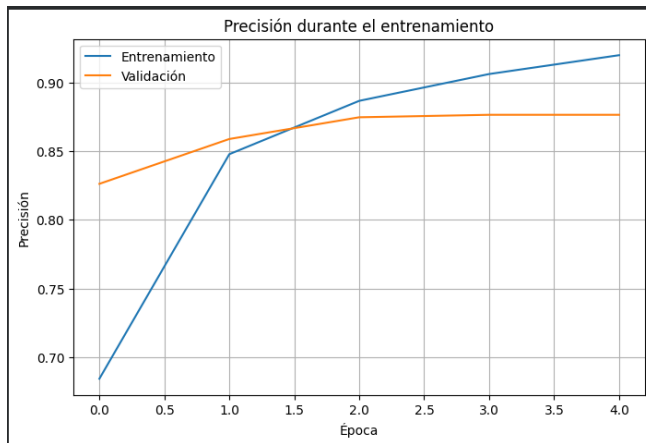
... Epoch 1/5
157/157 ————— 3s 11ms/step - accuracy: 0.6844 - loss: 0.6194 - val_accuracy: 0.8262 - val_loss: 0.4742
Epoch 2/5
157/157 ————— 2s 10ms/step - accuracy: 0.8478 - loss: 0.3847 - val_accuracy: 0.8588 - val_loss: 0.3395
Epoch 3/5
157/157 ————— 2s 10ms/step - accuracy: 0.8866 - loss: 0.2887 - val_accuracy: 0.8746 - val_loss: 0.3053
Epoch 4/5
157/157 ————— 3s 12ms/step - accuracy: 0.9061 - loss: 0.2449 - val_accuracy: 0.8764 - val_loss: 0.3029
Epoch 5/5
157/157 ————— 2s 11ms/step - accuracy: 0.9197 - loss: 0.2123 - val_accuracy: 0.8764 - val_loss: 0.3078

[7]
✓ 1s # 7. EVALUACIÓN
      test_loss, test_accuracy = model.evaluate(test_data, test_labels, verbose=0)

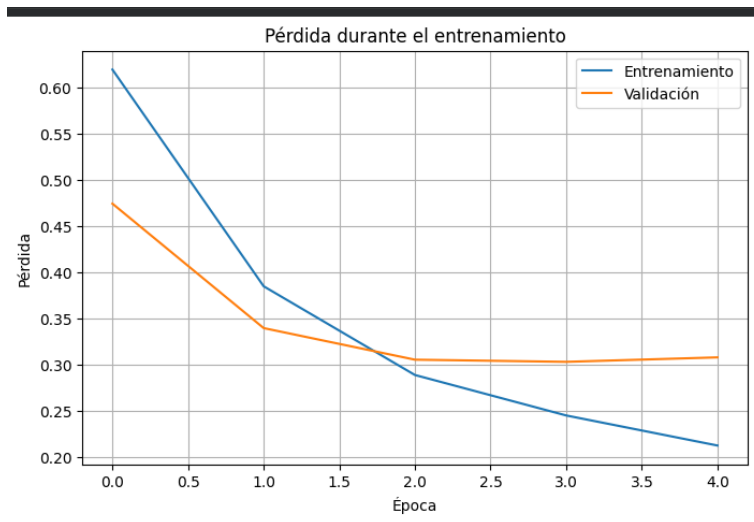
      print(f"Pérdida en prueba: {test_loss:.4f}")
      print(f"Precisión en prueba: {test_accuracy:.4f}")
      print(f"Precisión porcentual: {test_accuracy*100:.2f}%")

      Pérdida en prueba: 0.3306
      Precisión en prueba: 0.8609
      Precisión porcentual: 86.09%
```

- **Figura 4.** Gráfica de la precisión (*Accuracy*) durante el entrenamiento.



- **Figura 5.** Gráfica de la pérdida (*Loss*) en entrenamiento vs. validación.



- **Figura 6.** Pruebas del modelo clasificando textos nuevos.

```
# 13. PRUEBAS
ejemplos = [
    "excellent movie good acting",
    "boring bad movie waste time",
    "good story excellent",
    "bad boring terrible"
]

for texto in ejemplos:
    etiqueta, probabilidad = predecir(texto)
    print(f"Texto: {texto}")
    print(f"Predicción: {etiqueta} | Probabilidad positiva: {probabilidad:.3f}")
    print("." * 60)

*** Texto: excellent movie good acting
Predicción: positivo | Probabilidad positiva: 1.000
-----
Texto: boring bad movie waste time
Predicción: negativo | Probabilidad positiva: 0.000
-----
Texto: good story excellent
Predicción: positivo | Probabilidad positiva: 1.000
-----
Texto: bad boring terrible
Predicción: negativo | Probabilidad positiva: 0.000
-----
```

8. Repositorio de GitHub

- **Enlace al repositorio:** <https://github.com/stivenalvarez/proyecto-embeddings-tensorflow.git>
- **Notebook principal:** embeddings_tensorflow.ipynb

9. Conclusiones

- Este laboratorio me ayudó a entender en la práctica cómo se conecta el texto con los modelos matemáticos a través de tokens y vectores.
- La capa de Embedding demostró ser una pieza fundamental, ya que transformó números planos en vectores capaces de capturar el contexto de las reseñas.
- Lograr una precisión del 86.09% confirma que el modelo aprendió a generalizar correctamente con datos que nunca vio en la fase de entrenamiento.
- Analizar los resultados va más allá de ver si el código corre; implica comprender las métricas, identificar en qué casos el modelo puede fallar y evaluar el alcance real de la solución.

10. Referencias

Developers, T. (2022). TensorFlow. *Zenodo*.

Pang, B., Nijkamp, E., & Wu, Y. N. (2020). Deep learning with tensorflow: A review. *Journal of Educational and Behavioral Statistics*, 45(2), 227-248.